Healthcare Appointment System — Technical Feasibility

Oct 13, 2026 · @Hunny

1. Overview and Verdict

The Healthcare Appointment System is technically feasible with the MERN stack, using only mature, open-source tools and no specialised hardware. It serves three roles (patient, doctor, admin) through one React web app backed by a REST API and MongoDB. The hardest technical problem is preventing double-booking under concurrent requests, which MongoDB's unique indexes and atomic updates solve. This document covers technical feasibility only.

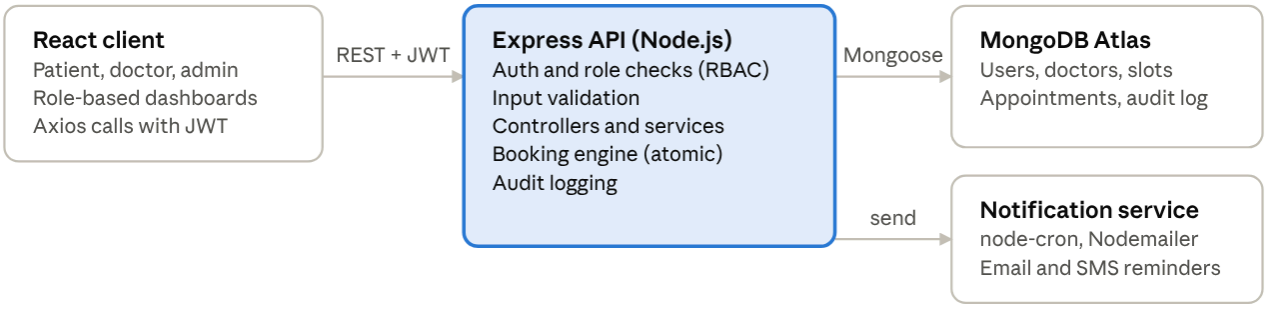
2. Technology Stack

Every component below is free, widely documented and production-proven.

Layer	Technology	Role
Frontend	React.js, React Router, Redux Toolkit/Context, Axios, Tailwind CSS	Role-based UI, routing, state, API calls
Backend	Node.js, Express.js	REST API, business logic, middleware
Database	MongoDB with Mongoose ODM	Schemas, validation, indexing
Auth	JWT, bcrypt	Stateless login, password hashing
Notifications	Nodemailer (email), node-cron (reminders), optional SMS gateway	Confirmations and reminders
Validation	Joi / express-validator	Input checking
Testing	Jest, Supertest, React Testing Library, Postman	Unit, API and UI tests
DevOps	Git/GitHub, Docker, GitHub Actions	Version control, containers, CI/CD
Hosting	Vercel/Netlify (frontend), Render/AWS (API), MongoDB Atlas	Cloud deployment

3. System Architecture

One React app talks to one stateless API backed by MongoDB



Stateless API: more instances can run behind a load balancer

system architecture · client, API, database, notifications

A three-tier design: the React client calls the Express API, which reads and writes MongoDB; a scheduler sends reminders independently of user requests.

4. Functional Requirements

Patient

- Register, log in, reset password, edit profile
- Search doctors by specialization, name or department
- View open time slots and book, cancel or reschedule
- Track appointment status and view history
- Receive confirmation and reminder notifications

Doctor

- Register (admin-approved), manage profile and qualifications
- Define weekly availability, slot duration, leave and breaks
- View a daily/weekly appointment dashboard
- Update status: Pending, Confirmed, Completed, Cancelled, No-show

Admin

- Manage users, doctors, departments and specializations
- Approve doctor accounts; activate or block users
- Monitor all appointments, and view reports and audit logs

System

- Prevent double-booking and past-time booking
- Generate slots from availability rules
- Send automated email/SMS reminders
- Enforce role-based access on every endpoint

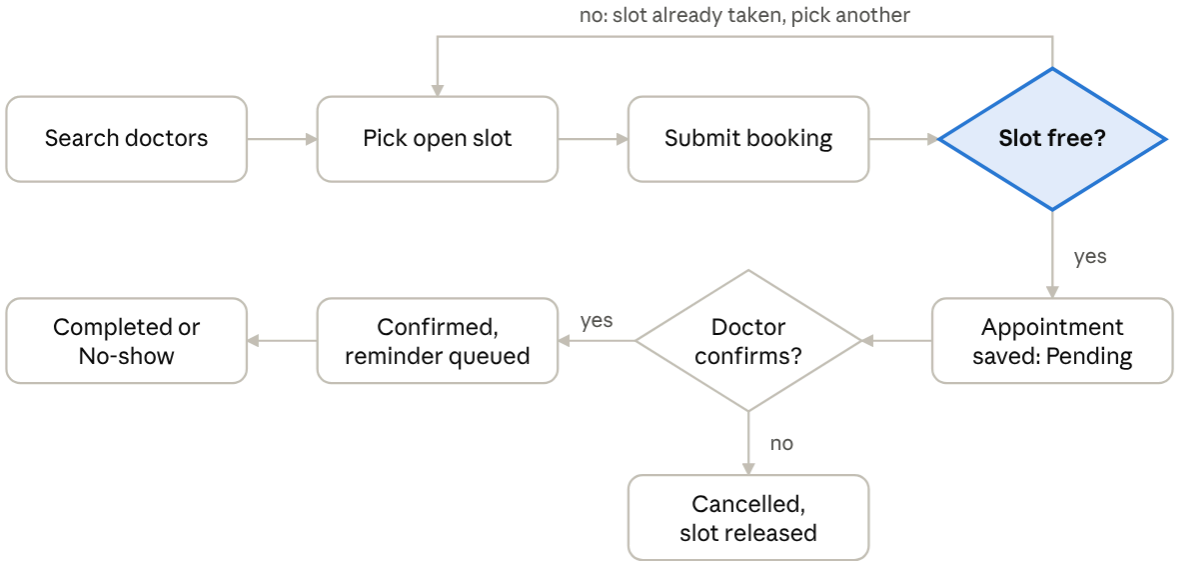
5. Non-Functional Requirements

Attribute	Requirement
Performance	API responses under 500 ms for typical calls; pages load in under 3 s; pagination and indexed queries
Scalability	Stateless API scales horizontally behind a load balancer; MongoDB Atlas scales by replica sets and sharding
Availability	Target 99.5%+ uptime; managed database with automated backups

Attribute	Requirement
Reliability	Atomic booking, transaction-safe cancellations, centralised error handling and logging
Security	Encryption, RBAC, validation (see Section 9)
Usability	Responsive (mobile/desktop) UI, clear booking flow, accessibility per WCAG basics
Maintainability	Modular MVC structure, ESLint/Prettier, documented REST API (Swagger)
Compatibility	Latest Chrome, Firefox, Edge and Safari
Auditability	Logged admin actions and appointment status changes

6. Booking Workflow

Only one booking can claim a slot



A patient cancel or reschedule also releases the slot and notifies the doctor

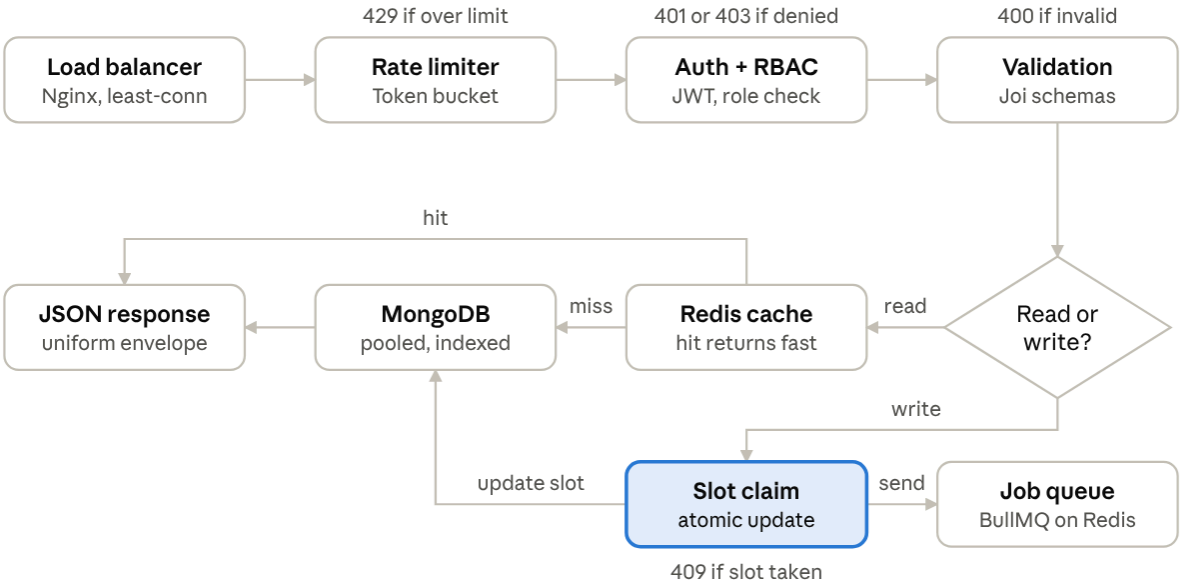
booking workflow · 2 decisions, 1 loop

The slot check is a single atomic database operation backed by a unique index, so two simultaneous requests cannot both succeed; the loser is asked to choose another slot.

7. API Request Handling and High-Concurrency Pipeline

A Node.js cluster can serve 1,000 concurrent users comfortably when requests pass through a load balancer, a token-bucket rate limiter, cache-aside reads, an atomic slot claim and a background job queue. Assuming 1,000 users each send one request every 5 seconds, peak load is about 200 requests per second; this is an estimate to confirm by load testing.

Four gates and a cache protect the database under load



Slow work (email, SMS) never blocks the response; failed jobs retry from the queue

request pipeline · 4 gates, cache, atomic claim, job queue

- **Load balancing:** Nginx with least-connections routing across 3 or more stateless API instances (PM2 cluster mode or Docker replicas); health checks remove failing instances.
- **Rate limiting:** token bucket per user and IP, stored in Redis (for example, a burst of 20 and a refill of 5 per second); excess requests get 429 with a Retry-After header.
- **Cache-aside reads:** doctor lists, specializations and slot availability are cached in Redis for 30 to 60 seconds; the slot cache is cleared on every booking.
- **Accuracy under concurrency:** the slot claim is an atomic compare-and-set (`findOneAndUpdate` where isBooked is false), backed by the unique slot index; an idempotency key on booking requests prevents duplicates when a client retries.
- **Asynchronous work:** confirmations and reminders go to a BullMQ queue with retries and exponential backoff, so the API responds first.
- **Overload protection:** MongoDB connection pooling, 5-second request timeouts, a circuit breaker on email/SMS, body-size limits, pagination and gzip. If the system saturates, it returns 503 (load shedding) instead of queuing without limit.

- **Stability:** graceful shutdown that drains open requests, a health endpoint, structured logs and CPU-based autoscaling. Validate with k6 or Artillery at 1,000+ virtual users.

Every response uses one envelope: success, data and error (code, message).

Code	Meaning
200 / 201	Success / resource created
400	Invalid input
401 / 403	Not logged in / role not allowed
404	Resource not found
409	Slot already booked
429	Rate limit exceeded
503	System overloaded, retry later

8. Database Structure (MongoDB)

Seven collections cover the system. References use ObjectIds; slots are stored as documents so booking can be atomic.

Collection	Key fields	Relations / indexes
users	name, email, passwordHash, role (patient/doctor/admin), phone, isActive, createdAt	Unique index on email
doctors	userId, specializationId, departmentId, qualification, experience, fee, bio, approved	userId → users; index on specializationId
specializations	name, description	Unique index on name
availabilities	doctorId, dayOfWeek, startTime, endTime, slotDuration, leaveDates	doctorId → doctors
slots	doctorId, date, startTime, endTime, isBooked	Unique compound index (doctorId, date, startTime)
appointments	patientId, doctorId, slotId, status, reason, notes, createdAt	Unique index on slotId for active bookings; index on (patientId, status)

Collection	Key fields	Relations / indexes
notifications	userId, type, message, sentAt, readStatus	userId → users

An audit log collection (actor, action, target, timestamp) records admin actions.

9. Security

- **Authentication:** bcrypt-hashed passwords; short-lived JWT access tokens with refresh tokens in httpOnly cookies; account lockout and rate limiting on login
- **Authorization:** role-based middleware (patient, doctor, admin) on every route; users can access only their own records
- **Transport and data:** HTTPS/TLS everywhere; MongoDB encryption at rest; secrets in environment variables
- **API hardening:** Helmet, CORS whitelist, input validation, NoSQL-injection sanitising (express-mongo-sanitize), XSS protection
- **Privacy:** minimal medical data stored; where applicable, align with regulations such as HIPAA or GDPR (consent, deletion, access logs)
- **Monitoring:** audit logs, error logging (Winston), automated daily backups

10. Resource Requirements

- **Hardware (development):** laptop with 8 GB RAM or more, modern browser; no special hardware
- **Software:** VS Code, Node.js LTS, MongoDB Atlas free tier, Postman, Git, Docker
- **Team skills:** React, Node/Express, MongoDB, REST design, basic DevOps and testing
- **Production:** one app server (2 vCPU, 4 GB RAM is enough to start), managed MongoDB cluster, domain and SSL certificate, email service

11. Technical Risks and Conclusion

Risk	Mitigation
Double-booking under concurrent requests	Unique slot index plus atomic update; reject on conflict
Sensitive data exposure	Encryption, RBAC, minimal data, audit logs
Reminder delivery failures	Retry queue; email fallback for SMS

Risk	Mitigation
Scaling load at peak hours	Stateless API, caching, indexes, horizontal scaling
Third-party service downtime	Graceful degradation and error logging

Conclusion: All required features map directly to standard MERN capabilities. No unproven technology or special hardware is needed, so the project is technically feasible.